

UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Allocare la memoria usando il free store

Stefano Ghidoni

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE



Agenda

- Acquisizione di memoria dal free store
- Puntatori e free store
- Garbage

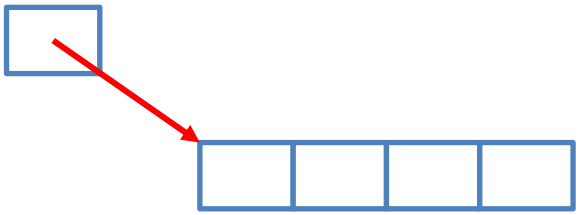
Allocazione dinamica

- L'allocazione dinamica della memoria permette di accedere al **free store**
  - Area di memoria molto ampia
  - Interazione col sistema operativo a tempo di esecuzione
    - Blocco di memoria noto a tempo di esecuzione
    - Problemi di efficienza
  - Nessun impatto sul record di attivazione e stack

Allocazione dinamica

- Come accedere al free store?
  - Usando new e i puntatori

```
double* p = new double[4];
```



- new usa il free store
- Questo procedimento prende il nome di **allocazione dinamica della memoria**

Puntatori e FS

- new ritorna elementi singoli o array
- Un puntatore **non sa** quanti oggetti sono presenti nell'array ritornato da new
- new ritorna un puntatore anche se allochiamo un solo elemento
  - Ritorna il puntatore all'elemento allocato o al primo elemento se è allocato un array
- La memoria ritornata da new **non ha un nome**
  - Memoria gestita tramite il puntatore restituito

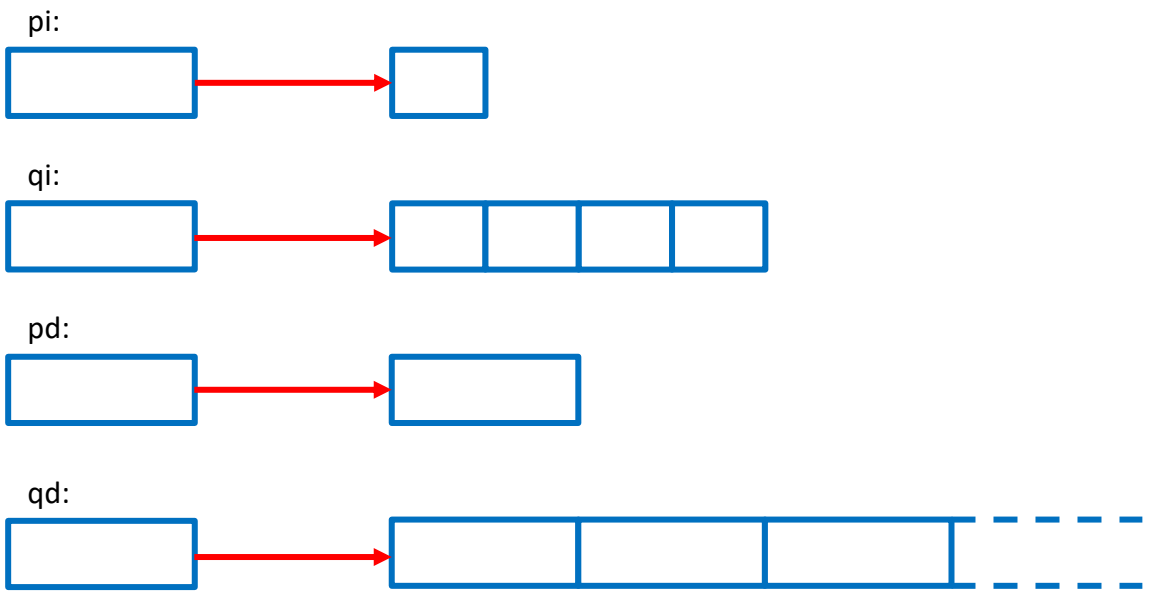
Puntatori e FS

- Esempi

```
int* pi = new int;
int* qi = new int[4];

double* pd = new double;
double* qd = new double[n];
```

Può essere una variabile



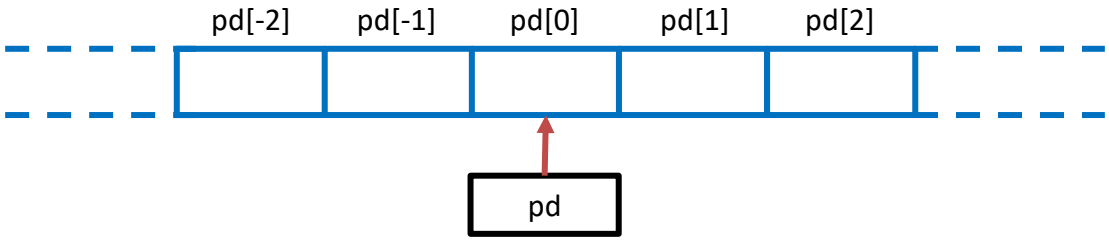
Accesso agli elementi

```
double* p = new double[4];
double x = *p;          // primo oggetto puntato
double y = p[2];        // terzo oggetto
```

- Nella gestione della memoria dinamica usiamo spesso:
  - Il nesso tra puntatori e array
  - L'aritmetica dei puntatori
- Ricoridamo che:
  - *p e p[0] sono equivalenti
  - Applicare [] a un puntatore equivale a vedere la memoria come un array

Range e range check

- A questo livello non esiste controllo sui range!
  - Simile agli array stile C




```
pd[2] = 2.2;          // ok!
pd[4] = 4.4;          // ok!
pd[-3] = -3.3;        // ok, ma...
```

"Il mio programma termina misteriosamente..."

- Grave problema
- Esecuzioni diverse possono dare sintomi diversi
- Bug molto difficili da gestire!

Inizializzazione

- Esempi di inizializzazione del puntatore e della memoria allocata

```
double* p0;                     // grosso problema!

double* p1 = new double;        // double non inizializzato

double* p2 = new double {5.5};      // double inizializzato

double* p3 = new double[5];   // array non inizializzato
```

Inizializzazione

- La memoria allocata con new non è inizializzata per i tipi built-in
- Per ovviare:

```
double* p4 = new double[5] {0, 1, 2, 3, 4}; // array init

double* p5 = new double[] {0, 1, 2, 3, 4};  // dimensione
                                            // dedotta
```

Inizializzazione con UDT

```
X* px1 = new X;

X* px2 = new X[17];
```

Chiamata al costruttore di default

- Se il costruttore di default non esiste (es., esiste solo un costruttore che accetta un int):

```
Y* py1 = new Y;          // errore

Y* py2 = new Y[17];      // errore

Y* py3 = new Y{13};      // ok

Y* py4 = new Y[17] {0, 2, 3, 4, ..., 16};   // ok
```

Controllo di validità

```
if (p0 != nullptr) {/* ... */}

if (p0) {/* ... */}
```

- Attenzione quando
  - Il puntatore non è mai stato inizializzato
    - Non è inizializzato a nullptr automaticamente!
  - Il puntatore si riferisce a memoria non valida (liberata)

Spostamento di un puntatore

- Un puntatore può essere spostato da un oggetto a un altro
  - Differenza con le reference!

```
double* p = new double;
double* q = new double[1000];

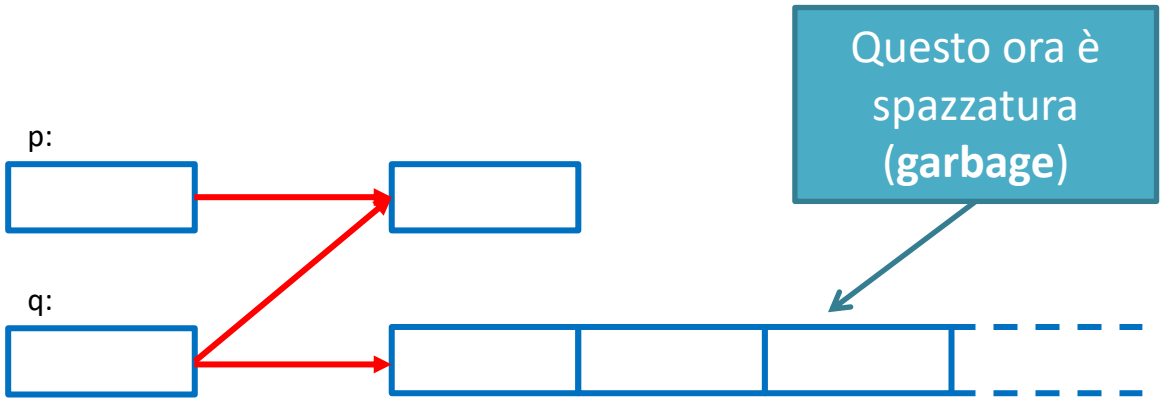
q[700] = 7.7;          // ok
q = p;                 // attenzione!!
double d = q[700];     // attenzione!!
```

- Questo può generare grossi problemi…

Spostamento di un puntatore

```
double* p = new double;
double* q = new double[1000];

q = p;                          // attenzione!!
```



Garbage

- Un'area di memoria di cui si perdono i riferimenti diventa **garbage**
- Tale zona di memoria rimane allocata ma è irraggiungibile e inservibile
- Questo fenomeno è noto come **memory leak**
  - Può portare all'esaurimento della memoria



Recap

- Creazione di strutture dinamiche come std::vector
- Allocazione dinamica della memoria
  - Accesso al free store
- Accesso agli elementi e range check
- Inizializzazione
- Garbage



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Allocare la memoria usando il free store

Stefano Ghidoni

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE